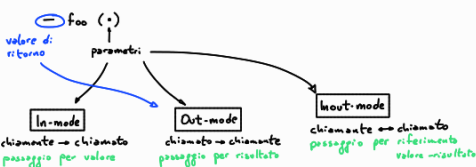


PASSAGGIO DI PARAMETRI



Passaggio per valore

I parametri attuali sono valutati (potenzialmente espressioni) e il valore ottenuto è copiato nei parametri formali:

=> modifiche al parametro formale non sono visibili alla procedura chiamante

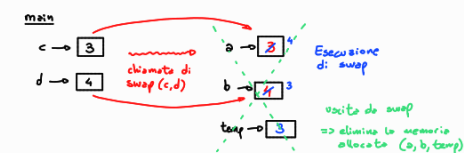
```
void foo (int x) { x := x + 1; }
...
y = 1;
foo(y);
```

Annotations:
- 'x diventa 3' points to the parameter x in the function definition.
- 'parametro formale allocato nello stack' points to the parameter x in the function definition.
- 'viene valutato y+1 = 2' points to the argument y in the function call.
- '2 viene usato per inizializzare x dentro foo' points to the argument y in the function call.
- 'y vale 1' points to the variable y in the main function.
- 'non c'è alcun legame tra la y del chiamante e la x del chiamato' points to the parameter x in the function definition.
- 'x viene eliminata all'uscita di foo' points to the parameter x in the function definition.

Esempio

```
void swap (value int a, value int b) {
    int temp = a;
    a = b;
    b = temp;
}

swap(c, d) // c=3, d=4
```



=> all'uscita di swap c e d non sono cambiati

Passaggio per riferimento

viene passato dal chiamante al chiamato l'indirizzo del parametro attuale

=> in-out mode

```
void foo (reference int x) { x := x + 1; }
...
y = 1;
foo(y);
```

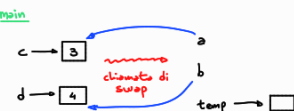
Annotations:
- 'prendiamo l'indirizzo (locazione) di y e lo associamo anche a x' points to the parameter x in the function definition.
- '=> x e' ATRAS di y' points to the parameter x in the function definition.
- 'l'indirizzo deve essere un l-value' points to the parameter x in the function definition.
- 'La modifica del parametro attuale modifica la cella puntata all'attuale' points to the parameter x in the function definition.

=> All'uscita dalla procedura eliminiamo il nome x ma non il suo contenuto riferibile da y

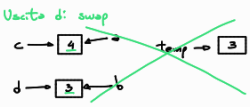
=> Tutte le modifiche al parametro formale sono modifiche al valore del parametro attuale

```
void swap (ref int a, ref int b) {
    int temp = a;
    a = b;
    b = temp;
}

swap(c, d);
// c=3, d=4
```

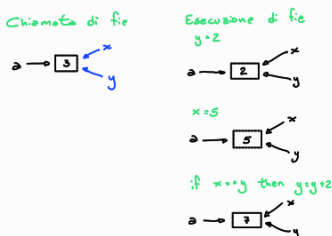


Esecuzione di swap

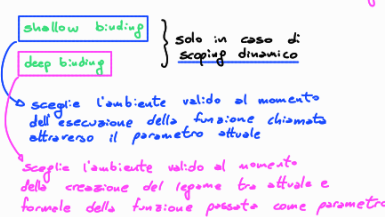
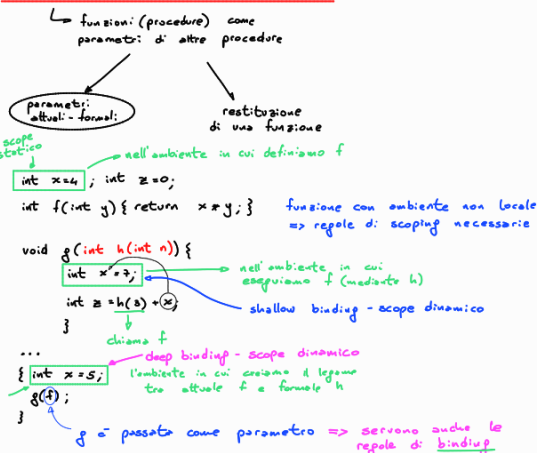


```
void fie (ref int x, ref int y) {
    y = 2; x = 5;
    if (x == y) then y = y + 2;
}
```

```
...
int a = 3;
fie(a, a);
```



FUNZIONI DI ORDINE SUPERIORE



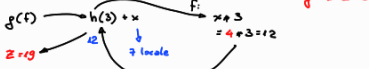
statico

```
1 int x=4; int z=0;
  if f(int y){ return x*y; }
  void g(int h(int n)){
    int x=7; 3 shallow + dinamico
    z=h(3)+x;
  }
```

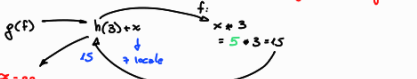
deep + dinamico

```
2 { int x=5; g(f) }
```

Scope statico: ambiente globale



Deep binding - scope dinamico



Shallow binding - scope dinamico

